Лабораторная работа 2 — часть «Реализация CART с нуля»

Аналитические и вычислительные задания

Содержание

Назначение работы 1

Результаты обучения 1

Данные, воспроизводимость и академическая честность 1

Как работать с ноутбуком 1

Короткая теория и соглашения 2
1. Узел и бинарное правило . . . 2
2. Неоднородность листа . . . 2
3. Кандидатные пороги . . . 2
4. Рекурсивный рост и остановка . . . 3
5. Вероятности и importance . . . 3

Последовательность заданий 3
0. Подготовка окружения и загрузка ARFF . . . 3
1. Разведочный анализ и честное разбиение . . . 3
2. Критерии и кандидатные разбиения . . . 4
3. Узел и рекурсивное дерево . . . 4
4. Контролируемый toy-тест . . . 5
5. Первое дерево и визуализация . . . 5
6. Выбор гиперпараметров по validation . . . 5
7. Единственная финальная оценка и сравнение со sklearn . . . 6
8. Краткий отчёт и самопроверка . . . 6

Критерии проверки 6

Источники 7

Назначение работы

В этой работе вы реализуете бинарный классификатор CART (*Classification and Regression Tree*) на NumPy, а затем сравните его с `sklearn.tree.DecisionTreeClassifier`. Цель — не получить готовую модель одной строкой, а проследить всю цепочку: от неупорядоченного исходного набора и критерия неоднородности до рекурсивного дерева, вероятностей в листьях и честной оценки на отложенной выборке.

Сквозной набор — Rice (Cammeo and Osmancik) из UCI: морфологические признаки отдельных зёрен риса и два сорта. В лабораторной используются только числовые признаки, поэтому можно сосредоточиться на механике CART. Это **не** пример системы контроля качества или пищевой экспертизы: в данных нет информации о партиях, приборе съёмки и цене ошибки.

Теоретическая опора — лекция о деревьях решений и ансамблях. Практические соглашения этой работы — фиксированное случайное состояние, раздельные train/validation/test и запрет использовать test для выбора модели — обязательны и важнее достижения высокой метрики.

Результаты обучения

После выполнения работы вы сможете:

- объяснить, почему CART строит бинарные разбиения и откуда берутся пороги;
- вычислять Gini и энтропию по меткам или числам объектов классов;
- выбирать разбиение по взвешенной неоднородности и уменьшению неоднородности;
- реализовать рекурсивное дерево с условиями остановки;
- получать метку, вероятность класса и importance признаков из собственного дерева;
- выбирать гиперпараметры по validation, а test использовать ровно для финальной оценки;
- сопоставлять собственную реализацию с деревом scikit-learn, не выдавая совпадение метрик за доказательство идентичности алгоритмов.

Данные, воспроизводимость и академическая честность

Исходные файлы находятся локально в `data/rice/`. Не скачивайте их из ноутбука: он должен запускаться без сети. Карточка данных содержит автора, лицензию CC BY 4.0, DOI, схему и контрольные суммы. При использовании набора в отчёте приведите ссылку на UCI Rice (Cammeo and Osmancik) и исходную работу Cinar и Koklu (2019).

Все случайные действия в этой работе используют `random_state=42`. ARFF-файл сгруппирован по метке класса; его нельзя разбивать по позиции строки. До формирования выборок необходимо явно перемешать строки, затем сделать стратифицированное разбиение:

$$\text{train} = 64\%, \qquad \text{validation} = 16\%, \qquad \text{test} = 20\%.$$

Классы кодируются после загрузки, а метка класса исключается из признаков. Нормализация числовых признаков для дерева не нужна: порог сравнивает значения одного и того же признака. Это не означает, что подготовка данных вообще не нужна — для категориальных признаков, пропусков и утечки нужны отдельные решения; они рассматриваются в части «Деревья и ансамбли в sklearn по CRISP-DM» Лабораторной работы 2.

Как работать с ноутбуком

Студенческий ноутбук содержит текст, интерфейсы, импорты и проверочные места. Заполняйте только незавершённые кодовые ячейки; Markdown-ячейки используйте для требуемых объяснений. Не открывайте решебник до сдачи: он существует для преподавательской проверки и самостоятельного разбора после попытки.

Пометки задают ожидаемый способ работы:

- **[Анализ]** — вывести формулу, проверить типы и размерности, сформулировать гипотезу;
- **[Python]** — реализовать или запустить вычисление;
- **[Проверка]** — добавить осмысленные `assert` и проверить инварианты;

- **[Интерпретация]** — объяснить результат словами, относящимися к данным и модели.

В коде используйте только `numpy`, `pandas`, `matplotlib`, `scipy` и `scikit-learn`. Основное дерево нельзя заменять `DecisionTreeClassifier`: sklearn разрешён лишь в заключительном сравнении. Не используйте `seaborn`, Graphviz или сетевые загрузки: так работа остаётся воспроизводимой в базовом окружении.

Короткая теория и соглашения

1. Узел и бинарное правило

Внутренний узел CART имеет вид

$$x_j \le t,$$

где j — индекс одного признака, t — порог. Объекты с истинным условием уходят в левое поддерево, остальные — в правое. Лист не делит данные дальше и хранит статистику обучающих объектов, попавших в него: количество каждого класса, наиболее частый класс и эмпирические вероятности классов.

Для бинарной классификации дерево разбивает пространство признаков на прямоугольные области. Один узел меняет только одну координату, поэтому даже сложная граница строится лестницей из простых правил. CART ищет **только бинарные** разбиения; это соглашение упрощает рекурсию и одинаково применимо к классификации и регрессии.

2. Неоднородность листа

Пусть в узле K классов, а p_k — доля класса k. В работе реализуются два критерия:

$$H_{\text{Gini}} = 1 - \sum_{k=1}^{K} p_k^2, \tag{1}$$

$$H_{\text{entropy}} = -\sum_{k=1}^{K} p_k \log_2 p_k. \tag{2}$$

Слагаемое $0 \log_2 0$ по определению принимается равным нулю. Чистый узел имеет неоднородность 0. В бинарном узле максимумы Gini и энтропии достигаются при равных долях классов, однако численные масштабы критериев различаются; сравнивать их значения напрямую не следует.

Если левый и правый потомки содержат n_L и n_R объектов, то качество правила оценивается взвешенной неоднородностью:

$$H_{\text{split}} = \frac{n_L}{n} H_L + \frac{n_R}{n} H_R, \qquad n = n_L + n_R. \tag{3}$$

Уменьшение неоднородности (gain) равно

$$\Delta H = H_{\text{parent}} - H_{\text{split}}. \tag{4}$$

Нужно выбрать допустимое правило с наибольшим ΔH. Взвешивание обязательно: разделение, которое идеально очищает два объекта и почти не улучшает большую часть данных, не должно выглядеть лучшим только из-за маленького чистого листа.

3. Кандидатные пороги

Для одного числового признака сначала отсортируйте значения:

$$v_{(1)} \le v_{(2)} \le \ldots \le v_{(n)}.$$

Между двумя различными соседними значениями можно поставить порог

$$t_i = \frac{v_{(i)} + v_{(i+1)}}{2}, \qquad v_{(i)} < v_{(i+1)}. \tag{5}$$

Другие пороги между теми же значениями дают то же разбиение обучающих объектов. Порог между одинаковыми значениями недопустим: он оставляет одинаковые значения по разные стороны правила. Допустимость дополнительно ограничивается `min_samples_leaf`: и слева, и справа должно остаться не меньше заданного числа объектов.

В реализации вам разрешается эффективно вычислять gain для всех позиций одной сортировки с помощью накопленных счётчиков классов. Но смысл должен остаться тем же: каждая допустимая позиция соответствует порогу из (5).

4. Рекурсивный рост и остановка

Рекурсивный метод получает объекты текущего узла, создаёт его статистику и делает одно из двух:

1. возвращает лист, если достигнута глубина, узел чист, объектов недостаточно или gain слишком мал;
2. сохраняет лучшее правило и рекурсивно строит левое и правое поддеревья.

В этой работе глубина корня равна нулю. Параметр `max_depth=0` поэтому означает дерево из одного листа. `min_samples_split` проверяется до поиска правила, а `min_samples_leaf` — для каждой стороны потенциального правила. Это разные ограничения.

5. Вероятности и importance

Для объекта, дошедшего до листа, вероятность класса k определяется частотой этого класса среди обучающих объектов листа:

$$\widehat{P}(y = k \mid x) = \frac{n_k^{\text{leaf}}}{n^{\text{leaf}}}. \tag{6}$$

Импортанс признака в самодельной модели определите как сумму $n_t \Delta H_t$ по всем внутренним узлам t, использующим этот признак. Затем нормируйте вектор так, чтобы при наличии разбиений сумма была равна единице. Это impurity-based importance: он отражает вклад в уменьшение критерия на train, но не устанавливает причинность и может перераспределяться между сильно коррелированными признаками.

Последовательность заданий

0. Подготовка окружения и загрузка ARFF

1. **[Python]** Импортируйте модули, задайте константу `RANDOM_STATE = 42` и путь к локальному ARFF через `pathlib.Path`.
2. **[Python]** Прочитайте ARFF `scipy.io.arff.loadarff`, превратите результат в `pandas.DataFrame` и декодируйте байтовые метки `Class` в строки. Не меняйте исходный файл.
3. **[Анализ]** До перемешивания посмотрите первые и последние метки. Объясните в Markdown, почему случайное разбиение по срезам строк здесь даёт некорректную оценку.
4. **[Python]** Выполните `sample(frac=1, random_state=RANDOM_STATE).reset_index(drop=True)`. Сохраните имена семи признаков отдельно от целевого столбца.
5. **[Проверка]** Проверьте число строк, число признаков, отсутствие пропусков, наличие только двух классов и отсутствие `Class` в матрице признаков. Проверки должны сообщать именно инвариант, а не печатать молчаливое число.

Ожидаемый тип результата: таблица с числовыми признаками и строковой целью; после перемешивания порядок строк отличается от исходного, но состав данных не меняется.

1. Разведочный анализ и честное разбиение

1. **[Python]** Постройте таблицу `describe()` и частоты классов. В Markdown отметьте, сбалансирован ли набор настолько, чтобы одной accuracy было достаточно для вывода.
2. **[Python]** Постройте гистограммы всех признаков, а для одной осмысленной пары — scatter-график с цветом по сорту. Подпишите единицы там, где они известны: `Area` и `Convex_Area` измерены в пикселях, `Perimeter` — в пикселях.

3. **[Интерпретация]** Назовите хотя бы одну пару признаков, которая может быть связана геометрически. Не объявляйте корреляцию причиной и не делайте вывод о сорте по одному графику.
4. **[Python]** Закодируйте `Class` в 0/1 и сохраните отображение к исходным названиям классов. Разделите данные дважды через `train_test_split`: сначала выделите test, затем validation из временной train-части. В обоих вызовах используйте `stratify` и `random_state=42`.
5. **[Проверка]** Выведите размеры трёх выборок и долю положительного класса в каждой. Проверяйте, что индексы/строки не пересекаются и сумма размеров совпадает с размером исходного набора.

Ожидаемый тип результата: три непересекающихся набора приблизительно в пропорции 64/16/20, каждый с близкой долей классов. Test отложите до раздела 7.

2. Критерии и кандидатные разбиения

1. **[Анализ]** На маленьком массиве меток вручную выпишите числа объектов каждого класса, Gini и энтропию. Затем перенесите вычисление в Markdown-ячейку.
2. **[Python]** Реализуйте `class_counts(y, n_classes)`, `gini_from_counts(counts)` и `entropy_from_counts(counts)`. Функции должны работать как для одного вектора счётчиков, так и для матрицы счётчиков по нескольким кандидатам. Обработайте нулевые вероятности без предупреждений и `nan`.
3. **[Проверка]** Добавьте проверки для чистого, смешанного и пустого узла. Для непустого узла оба критерия неотрицательны; для чистого — нулевые.
4. **[Python]** Реализуйте `candidate_thresholds(values)`: отсортируйте уникальные значения и верните середины между соседними различными значениями. Для менее чем двух уникальных значений возвращайте пустой массив корректного числового типа.
5. **[Python]** Реализуйте `weighted_impurity(left_counts, right_counts, criterion)` и `best_split(X, y, n_classes, criterion, min_samples_leaf)`. Последняя функция должна вернуть индекс признака, порог, gain и булеву маску левой ветви либо явный признак отсутствия допустимого правила.
6. **[Проверка]** На toy-наборе проверьте, что выбранный порог отделяет объекты, обе стороны удовлетворяют `min_samples_leaf`, gain положителен, а взвешенная неоднородность не превосходит родительскую с разумной численной погрешностью.

Подсказка: сортировка одного столбца задаёт порядок объектов. Накопленные количества классов слева и полные количества минус левые справа позволяют получить критерии для всех границ между соседними значениями без повторной фильтрации всей матрицы.

3. Узел и рекурсивное дерево

1. **[Python]** Опишите `TreeNode` через `@dataclass`: глубина, количество объектов, вектор количеств классов, предсказание, вероятности, неоднородность, а также необязательные `feature_index`, `threshold`, `left`, `right`. Добавьте понятное свойство `is_leaf`.
2. **[Python]** Создайте класс `CARTClassifier` с параметрами `criterion`, `max_depth`, `min_samples_split`, `min_samples_leaf` и `min_impurity_decrease`. В `fit` проверьте формы `X` и `y`, закодируйте классы, сохраните число признаков и вызовите рекурсивный метод. Имена признаков передаются в функцию визуализации отдельно: модель получает обычную числовую матрицу.
3. **[Python]** В рекурсивном методе сначала создавайте статистику текущего узла. Затем последовательно учитывайте чистоту узла, лимит глубины, минимальное число объектов для поиска, отсутствие допустимого правила и минимальное уменьшение критерия. Только после этого создавайте дочерние узлы.
4. **[Python]** Реализуйте `predict_proba`: для каждой строки проходите от корня по условию `<= threshold` до листа и возвращайте сохранённые вероятности. `predict` выбирает класс с максимальной вероятностью и возвращает исходные, а не перекодированные метки.
5. **[Python]** Во время принятия разбиения накапливайте ненормированный вклад признака. После роста дерева сформируйте `feature_importances_`; при дереве без разбиений корректно возвращайте нулевой вектор, а не делите на ноль.

6. **[Проверка]** Убедитесь, что сумма вероятностей в каждой строке равна единице, все предсказания принадлежат `classes_`, глубина и число листьев согласованы с ограничениями, а importance неотрицательны и нормированы при наличии разбиений.

Ожидаемый тип результата: объект с публичными атрибутами, похожими по смыслу на sklearn (`classes_`, `feature_importances_`, `predict`, `predict_proba`), но с прозрачной собственной логикой.

4. Контролируемый toy-тест

1. **[Python]** Обучите неглубокое дерево на маленьком одномерном наборе, где класс меняется по значению признака. Проверьте выбранный порог, метки и вероятности.
2. **[Python]** Повторите опыт с `criterion="entropy"`. Сравните выбранное правило и численные значения критериев, не ожидая, что сами значения совпадут с Gini.
3. **[Python]** Добавьте тест для постоянного признака и тест для ограничения `min_samples_leaf`, при котором допустимого разбиения быть не должно.
4. **[Интерпретация]** Объясните, какие тесты проверяют алгоритм, а какие — только конкретный набор Rice. Укажите один инвариант, который не выражается высокой accuracy.

Эти маленькие проверки выполняются до обучения на реальном наборе: так ошибка в рекурсии или в формулах не маскируется хорошим случайным результатом на большом наборе.

5. Первое дерево и визуализация

1. **[Python]** Обучите базовое дерево только на train с умеренным ограничением глубины и минимального листа. Не меняйте параметры, опираясь на test.
2. **[Python]** Напишите функцию `plot_tree` на matplotlib. Внутренний узел должен показывать имя признака, знак сравнения, порог, число объектов и неоднородность; лист — число объектов, предсказываемый класс и вероятности. Нарисуйте дерево так, чтобы подписи не перекрывались.
3. **[Python]** Постройте столбчатую диаграмму `feature_importances_` с именами признаков. Отдельно покажите матрицу ошибок на validation.
4. **[Интерпретация]** Сформулируйте один путь от корня к листу как правило «если … и …, то …». Отделите описание правила от вывода о причинности признака.

Ожидаемый тип результата: читаемая схема ограниченного дерева и диаграмма, у которой подписи признаков совпадают с данными, а не с индексами столбцов.

6. Выбор гиперпараметров по validation

1. **[Python]** Составьте компактную сетку из `criterion` (`gini`, `entropy`), `max_depth` и `min_samples_leaf`. Изменяйте только эти параметры; `min_samples_split` оставьте согласованным с размером листа.
2. **[Python]** Для каждой конфигурации создайте новый объект, обучите только на train и оцените accuracy и balanced accuracy только на validation. Сохраните параметры, число листьев, глубину и метрики в DataFrame.
3. **[Python]** Отсортируйте результаты по основной метрике с явным вторичным правилом при равенстве. Выберите единственную конфигурацию по validation и зафиксируйте её до открытия test.
4. **[Интерпретация]** Опишите компромисс между сложностью дерева, качеством на train и качеством на validation. Не делайте выбор по одной случайной строке или только по train accuracy.

Ожидаемый тип результата: компактная таблица экспериментов и один выбранный набор параметров. Эта таблица — журнал решения, а не поиск «максимального числа» на test.

7. Единственная финальная оценка и сравнение со sklearn

1. **[Python]** Переобучите выбранную самодельную конфигурацию на объединении train и validation. Это разрешено, потому что параметры уже зафиксированы. Оцените её на test один раз: accuracy, balanced accuracy, classification report и confusion matrix.
2. **[Python]** Создайте `DecisionTreeClassifier` с теми же доступными параметрами и `random_state=42`; обучите на тех же объединённых данных и оцените на том же test. Не передавайте test в `fit`, `GridSearchCV` или параметры.
3. **[Python]** Сравните размеры деревьев, test-метрики, прогнозы вероятностей нескольких объектов и нормированные importances. При необходимости используйте `sklearn.tree.export_text` только как дополнительное текстовое представление.
4. **[Проверка]** Убедитесь, что тестовые метки не использовались при выборе `best_config`; сравните значения `classes_` и формы матриц вероятностей.
5. **[Интерпретация]** Объясните возможное расхождение результатов: разные соглашения при равных gain, детали остановки, численная точность или отличие поддерживаемых sklearn-параметров. Не объявляйте одну реализацию «неправильной» лишь по малому расхождению.

8. Краткий отчёт и самопроверка

В конце ноутбука заполните модельную карточку из 8–12 предложений:

1. какая задача и единица наблюдения;
2. откуда взяты данные, лицензия и ключевые ограничения;
3. как исключено влияние упорядоченного исходного файла;
4. размеры и назначение train/validation/test;
5. выбранный критерий и гиперпараметры;
6. финальные test-метрики без округления до целого процента;
7. один интерпретируемый путь дерева и один важный признак с оговоркой;
8. сравнение с sklearn и причина возможных расхождений;
9. минимум одно ограничение набора и одно ограничение impurity importance;
10. что потребовалось бы проверить до применения модели вне лабораторной.

Перед сдачей перезапустите ядро и выполните ноутбук сверху вниз. Сдаваемая версия должна содержать ваши объяснения, код, графики, проверки и итоговую карточку. Не включайте в неё сетевую загрузку, абсолютные пути, результаты из чужого запуска или test-метрики, использованные для подбора параметров.

Критерии проверки

Компонент	Что проверяется
Загрузка и EDA	локальный ARFF, декодирование, явное перемешивание, честный стратифицированный split, содержательная интерпретация
Математика split	корректные Gini/entropy, пороги, взвешивание, gain и ограничения листа
Реализация CART	рекурсивный `TreeNode`/класс, остановки, `predict`, `predict_proba`, публичные атрибуты и проверки
Эксперименты	validation для выбора, test только в конце, читаемые таблицы и визуализации
Анализ	обоснованные выводы, ограничения данных и importance, корректное сравнение со sklearn
Воспроизводимость	`random_state=42`, относительные пути, выполнение сверху вниз без интернета

Источники

1. UCI Machine Learning Repository: Rice (Cammeo and Osmancik), DOI 10.24432/C5MW4Z, CC BY 4.0.
2. Cinar, I.; Koklu, M. (2019). *Classification of Rice Varieties Using Artificial Intelligence Methods*. DOI 10.18201/ijisae.2019355381.
3. Документация scikit-learn: DecisionTreeClassifier.
4. Документация scikit-learn: дерево решений и importance признаков.